

Análise e Previsão de Atrasos em Voos Domésticos nos EUA

Ciência de Dados — Relatório Final (Parte 2)

Micael Gonçalves

Faculdade de Ciências Exatas e da Engenharia

Universidade da Madeira

Funchal, Portugal

2033117@student.uma.pt

Resumo—Este relatório apresenta as três últimas fases do ciclo de vida de análise de dados aplicadas ao *Flight Delay and Cancellation Dataset* (2019–2023). Partindo do trabalho realizado na Parte 1 — pré-processamento, análise exploratória, testes de hipóteses e engenharia de *features* — esta segunda parte foca-se na construção, avaliação e comparação de múltiplos modelos de *machine learning* para a previsão de atrasos em voos domésticos norte-americanos. Os modelos implementados incluem um algoritmo kNN desenvolvido de raiz em NumPy, dois modelos supervisionados via *scikit-learn*, dois modelos de *ensemble* (bagging e boosting), uma rede neuronal profunda em PyTorch, e dois algoritmos de clustering. Os resultados mostram que os modelos de *ensemble* alcançam o melhor desempenho preditivo, e que a previsão de atrasos antes da partida é intrinsecamente difícil dada a elevada variabilidade operacional do dataset.

Index Terms—atrasos em voos, machine learning, kNN, random forest, gradient boosting, deep learning, clustering, Python

I. INTRODUÇÃO

Este relatório constitui a continuação do trabalho desenvolvido na Parte 1, cobrindo as fases 4, 5 e 6 do ciclo de vida de análise de dados: construção de modelos, comparação e avaliação, e operacionalização.

O dataset utilizado é o *Flight Delay and Cancellation Dataset* (2019–2023) [1], derivado do sistema de reporte de pontualidade do Departamento de Transportes dos EUA. O dataset contém aproximadamente 2,9 milhões de registos de voos domésticos comerciais após pré-processamento. Todas as colunas com informação posterior ao evento (data leakage) foram removidas na Parte 1, garantindo que os modelos apenas utilizam informação disponível antes da partida.

As tarefas de modelação abordadas são:

- **Regressão:** previsão do atraso na chegada em minutos (ARR_DELAY).
- **Classificação:** categorização em *on-time* (< 15 min), atraso curto (15–30 min) e atraso longo (> 30 min).
- **Clustering:** identificação de padrões operacionais em voos e companhias aéreas.

II. FASE 4: CONSTRUÇÃO DE MODELOS

A. Preparação dos Dados para Modelação

O dataset final utilizado para modelação foi carregado diretamente do ficheiro CSV produzido na fase de engenharia de *features* da Parte 1, contendo 43 colunas que incluem variáveis escaladas, *features* temporais e de interação, e codificações categóricas. Antes do treino, as colunas de *target* e variáveis redundantes foram removidas conforme definido no ficheiro de configuração YAML. Os valores em falta remanescentes foram imputados com a mediana de cada coluna.

A divisão treino/teste foi realizada com uma proporção de 80%/20% e *random_state* fixo para garantir reprodutibilidade. Dado o volume de dados disponível (≈2,9M registos), esta abordagem foi preferida à validação cruzada, cujo custo computacional seria proibitivo para os modelos mais pesados.

B. kNN Implementado de Raiz

O algoritmo k-Nearest Neighbors foi implementado integralmente em NumPy, sem recurso à biblioteca *scikit-learn* para o núcleo do algoritmo. A implementação suporta duas modalidades:

- **Regressão:** previsão de ARR_DELAY em minutos, pela média dos k vizinhos mais próximos.
- **Classificação:** previsão da classe de atraso, por votação maioritária entre os k vizinhos.

A métrica de distância utilizada é a distância Euclidiana:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (1)$$

Para a regressão foram testados os valores $k \in \{13, 15, 17, 19, 21, 23, 25, 27, 29\}$ sobre uma amostra de 30.000 registos. Para a classificação, dado o problema de desequilíbrio de classes, foram utilizados valores menores $k \in \{3, 5, 7, 9, 11, 13, 15\}$ e as classes foram balanceadas por *undersampling* da classe maioritária, garantindo igual representação das três categorias. Os resultados de regressão são apresentados na Tabela I e os de classificação na Tabela II.

Os resultados de regressão mostram uma melhoria gradual com o aumento de k , sendo o melhor desempenho obtido

Tabela I
RESULTADOS DE REGRESSÃO KNN

k	MAE	RMSE	R ²
13	20.45	46.83	-0.077
17	20.34	46.45	-0.059
23	20.24	46.06	-0.042
27	20.16	45.90	-0.035
29	20.16	45.82	-0.031

Tabela II
RESULTADOS DE CLASSIFICAÇÃO KNN (CLASSES BALANCEADAS)

k	Accuracy	Macro F1	Weighted F1
3	0.473	0.474	0.474
5	0.442	0.439	0.440
7	0.428	0.426	0.426
9	0.421	0.420	0.421
11	0.424	0.424	0.424
13	0.424	0.423	0.423
15	0.402	0.402	0.402

com $k = 29$ (MAE ≈ 20.16 , R² ≈ -0.031). No entanto, todos os modelos kNN produziram R² negativos, indicando que o preditor baseado em distância não supera uma baseline de média simples. Isto deve-se à elevada variabilidade e dimensionalidade do dataset — fenómeno conhecido como *curse of dimensionality* — que torna as distâncias euclidianas menos significativas em espaços de alta dimensão.

Na classificação com classes balanceadas, o melhor resultado foi obtido com $k = 3$ (accuracy $\approx 47.3\%$, Macro F1 ≈ 0.474). Este resultado está acima da chance aleatória (33.3% com 3 classes equiprováveis), com desempenho consistente entre as três classes como confirmado pelas métricas macro e weighted praticamente idênticas. A utilização de k pequeno é justificada pelo facto de fronteiras de decisão mais locais produzirem melhores resultados quando as classes têm distribuições sobrepostas.

C. Modelos Supervisionados

Foram implementados dois modelos supervisionados adicionais via *scikit-learn*:

1) *Decision Tree Regressor*: Modelo baseado em regras de decisão, treinado sobre o dataset completo com profundidade máxima de 10 e mínimo de 50 amostras por folha. Não requer escalonamento de *features*.

2) *Support Vector Regressor (SVR)*: Modelo de regressão por vetores de suporte com *kernel* RBF, $C = 10$ e $\epsilon = 1$. Requer escalonamento, aplicado via *Pipeline* com *StandardScaler*. Treinado sobre uma amostra de 20.000 registos por limitações computacionais.

Os resultados são apresentados na Tabela III.

O Decision Tree produziu o melhor equilíbrio com R² positivo (≈ 0.014) e RMSE competitivo. O SVR apresentou o MAE mais baixo (13.54), mas com R² negativo, o que indica que, apesar de reduzir o erro médio em muitas amostras, o modelo não explica adequadamente a variância total dos atrasos. Este comportamento está parcialmente relacionado

Tabela III
RESULTADOS DOS MODELOS SUPERVISIONADOS

Modelo	MAE	RMSE	R ²
Decision Tree Regressor	19.28	47.77	0.014
Support Vector Regressor	13.54	51.18	-0.055

com a função de perda *epsilon-insensitive* do SVR ($\epsilon = 1$), que ignora erros inferiores a 1 minuto, e com o tamanho de amostra reduzido utilizado no treino. A comparação direta de MAE entre SVR e os restantes modelos deve ser interpretada com cautela.

D. Modelos de Ensemble

Foram implementados dois modelos de *ensemble*:

1) *Random Forest Regressor (Bagging)*: O Random Forest treina múltiplas árvores de decisão independentemente em sub-amostras aleatórias dos dados e das *features* (bootstrap), combinando as previsões pela média. Este mecanismo reduz a variância em relação a uma única árvore. Configuração: 100 estimadores, profundidade máxima 15, mínimo de 20 amostras por folha.

2) *Gradient Boosting Regressor (Boosting)*: O Gradient Boosting treina árvores sequencialmente, onde cada nova árvore tenta corrigir os erros residuais do ensemble anterior. Configuração: 150 estimadores, *learning rate* 0.05, profundidade máxima 3, mínimo de 30 amostras por folha.

Ambos os modelos foram treinados no mesmo conjunto de treino para garantir comparação justa. Os resultados são apresentados na Tabela IV.

Tabela IV
RESULTADOS DOS MODELOS DE ENSEMBLE

Modelo	MAE	RMSE	R ²
Random Forest (Bagging)	19.17	47.64	0.020
Gradient Boosting (Boosting)	19.30	47.72	0.017

Os modelos de *ensemble* produziram os melhores resultados de regressão. O Random Forest obteve o melhor R² (≈ 0.020) e o menor RMSE (≈ 47.64), superando ligeiramente o Gradient Boosting. Ambos melhoraram em relação à árvore de decisão individual, confirmando o benefício da combinação de múltiplos modelos.

E. Modelo de Deep Learning

Foi implementada uma rede neuronal *feed-forward* (MLP) em PyTorch para a tarefa de regressão. A arquitetura consiste em:

- Camada de entrada com dimensão igual ao número de *features*
- Camada densa: 128 neurónios, ativação ReLU, Dropout(0.2)
- Camada densa: 64 neurónios, ativação ReLU, Dropout(0.2)
- Camada de saída: 1 neurónio (regressão, sem ativação)

O modelo foi treinado com o otimizador Adam (*learning rate* 0.001), função de perda MSE, *batch size* de 512, e *early stopping* com paciência de 5 épocas. Foi utilizada uma amostra de 200.000 registos. Os resultados são apresentados na Tabela V.

Tabela V
RESULTADOS DO MODELO DE DEEP LEARNING

Modelo	MAE	RMSE	R ²
PyTorch MLP	19.36	51.92	0.011

A rede neuronal obteve um R² ligeiramente positivo (≈ 0.011), superando a baseline de média simples, mas não superou os modelos de *ensemble* baseados em árvores. Este resultado é consistente com a literatura: modelos de *deep learning* frequentemente não superam métodos clássicos em datasets tabulares estruturados sem arquiteturas especializadas ou maior volume de dados.

F. Clustering

Foram aplicados dois algoritmos de clustering sobre uma amostra de 100.000 voos, utilizando as *features*: ARR_DELAY, DISTANCE, CRS_ELAPSED_TIME, CRS_DEP_TIME_std, MONTH, DAY_OF_WEEK e dep_hour_x_elapsed. Dado que o clustering é uma tarefa não supervisionada de descoberta de padrões, a utilização de ARR_DELAY como *feature* não constitui *data leakage*.

1) *KMeans*: O algoritmo KMeans foi avaliado para $k \in \{2, \dots, 8\}$ utilizando inércia (método do cotovelo) e *silhouette score*. Os resultados são apresentados na Tabela VI.

Tabela VI
KMEANS: INÉRCIA E SILHOUETTE SCORE POR K

k	Inércia	Silhouette Score
2	543234.3	0.246
3	482659.3	0.171
4	437290.1	0.170
5	396102.5	0.171
6	362645.7	0.169
7	338328.6	0.170
8	313882.9	0.184

O melhor *silhouette score* foi obtido para $k = 2$ (≈ 0.246), resultando em dois grupos operacionais:

- Cluster 0: ≈ 36.529 voos
- Cluster 1: ≈ 63.471 voos

Estes clusters refletem provavelmente comportamentos operacionais distintos como voos de curta vs longa distância ou padrões de pontualidade diferenciados.

2) *DBSCAN*: O DBSCAN foi configurado com $\varepsilon = 0.8$ e *min_samples*=15. Os resultados identificaram 7 clusters com $\approx 2.98\%$ de pontos de ruído. A grande maioria das observações concentrou-se num único cluster denso (Cluster 0: ≈ 92.297 voos), com clusters muito pequenos que representam comportamentos operacionais atípicos ou anomalias.

3) *Clustering por Companhia Aérea*: Adicionalmente, foi realizado clustering ao nível da companhia aérea, agregando o dataset por AIRLINE_CODE e calculando métricas de desempenho por operador: atraso médio, desvio padrão do atraso, distância média, taxa de pontualidade e taxa de atraso longo. O KMeans foi aplicado sobre os perfis resultantes, com o número ótimo de clusters determinado por *silhouette score*.

Esta análise permite identificar grupos de companhias aéreas com comportamentos operacionais similares, respondendo diretamente ao requisito do projeto de clustering baseado em perfis de operadores.

III. FASE 5: COMPARAÇÃO E AVALIAÇÃO DE MODELOS

A. Comparação Global de Regressão

A Tabela VII apresenta a comparação de todos os modelos de regressão implementados, incluindo a baseline de regressão linear da Parte 1.

Tabela VII
COMPARAÇÃO GLOBAL DOS MODELOS DE REGRESSÃO

Modelo	MAE	RMSE	R ²
kNN (k=29)	20.62	59.07	-0.024
Decision Tree	19.28	47.77	0.014
SVR*	13.54	51.18	-0.055
Gradient Boosting	19.30	47.72	0.017
Random Forest	19.17	47.64	0.020
PyTorch MLP	19.36	51.92	0.011
Linear Regression†	19.47	47.86	0.011

* Treinado em amostra de 20k registos; MAE não comparável diretamente.

† Modelo baseline da Parte 1.

O Random Forest obteve o melhor desempenho global, com o menor RMSE (47.64) e o maior R² (0.020). É relevante notar que todos os modelos apresentam R² muito baixo, o que é esperado dado que as variáveis de *data leakage* (como DEP_DELAY) foram removidas. Os modelos operam exclusivamente com informação pré-partida, que tem poder preditivo limitado sobre a variância total dos atrasos.

B. Comparação de Classificação

Tabela VIII
COMPARAÇÃO DOS MODELOS DE CLASSIFICAÇÃO

Modelo	Accuracy	Macro F1
kNN Classificação (k=3, balanceado)	0.473	0.474

A classificação kNN com classes balanceadas atingiu 47.3% de *accuracy* com Macro F1 de 0.474, claramente acima da chance aleatória de 33.3% para 3 classes equiprováveis. O balanceamento por *undersampling* foi essencial para obter um resultado honesto — sem balanceamento, o modelo atingia 80.5% de *accuracy* mas com Macro F1 de apenas 0.297, prevendo sistematicamente a classe maioritária *on-time*. O valor $k = 3$ produziu o melhor resultado, confirmando que fronteiras de decisão locais são mais adequadas para este problema de classificação com classes sobrepostas.

C. Comparação de Clustering

Tabela IX
COMPARAÇÃO DOS ALGORITMOS DE CLUSTERING

Algoritmo	Resultado Principal	Silhouette
KMeans	Melhor k=2	0.246
DBSCAN	7 clusters, 2.98% ruído	N/A

Os valores de *silhouette* relativamente baixos (≈ 0.25) indicam que a separação entre clusters é moderada, o que é expectável dado que os padrões operacionais de voos se sobrepõem consideravelmente entre companhias, rotas e épocas do ano.

D. Pontos Fortes e Fracos de Cada Abordagem

1) *kNN de Raiz*: **Pontos fortes**: implementação transparente, compreensão direta do mecanismo de aprendizagem baseado em distância, não requer fase de treino explícita.

Pontos fracos: custo computacional elevado na previsão ($O(n)$ por amostra de teste), sensível ao escalonamento de *features*, desempenho fraco em dados de alta dimensionalidade e com desequilíbrio de classes.

2) *Decision Tree*: **Pontos fortes**: interpretável, captura relações não-lineares, fornece importância de *features*, não requer escalonamento.

Pontos fracos: sensível a *overfitting*, variância elevada em datasets ruidosos.

3) *SVR*: **Pontos fortes**: capacidade para modelar relações não-lineares via *kernel* RBF, propriedades teóricas de generalização sólidas.

Pontos fracos: computacionalmente dispendioso em datasets grandes, sensível a hiperparâmetros, MAE obtido não é diretamente comparável por ter sido treinado numa amostra menor.

4) *Random Forest*: **Pontos fortes**: melhor desempenho geral, robusto a ruído e *overfitting*, fornece importância de *features*, paralelizável.

Pontos fracos: menor interpretabilidade do que uma árvore simples, custo de memória elevado para grandes *ensembles*.

5) *Gradient Boosting*: **Pontos fortes**: correção sequencial de erros, bom desempenho em datasets tabulares.

Pontos fracos: sensível a hiperparâmetros, treino mais lento, risco de *overfitting* se o boosting for demasiado agressivo.

6) *Deep Learning (MLP)*: **Pontos fortes**: capacidade para aprender relações não-lineares complexas, arquitetura flexível. **Pontos fracos**: não superou os modelos de *ensemble* neste dataset tabular, requer ajuste cuidadoso de hiperparâmetros, menos interpretável.

7) *Clustering*: **Pontos fortes**: revelou agrupamentos operacionais sem necessidade de labels, identificou anomalias via DBSCAN.

Pontos fracos: separação de clusters moderada, resultados sensíveis à escolha de parâmetros.

E. Principais Conclusões

A análise demonstrou que a previsão de atrasos em voos é intrinsecamente difícil quando restrita a informação pré-partida. As principais conclusões são:

- Os modelos de *ensemble* (Random Forest e Gradient Boosting) alcançaram o melhor desempenho preditivo em regressão.
- O Random Forest produziu o melhor equilíbrio entre robustez e capacidade preditiva.
- O kNN de raiz demonstrou os princípios do aprendizado baseado em instâncias, mas foi superado por todos os outros modelos.
- O balanceamento de classes por *undersampling* foi essencial para obter resultados de classificação honestos, com o kNN a atingir 47.3% de *accuracy* e Macro F1 de 0.474 após balanceamento.
- O *deep learning* não superou os métodos clássicos de *ensemble* neste dataset tabular estruturado.
- O clustering revelou agrupamentos operacionais amplos, mas com sobreposição significativa, refletindo a natureza multifatorial dos atrasos em voos.

F. Recomendações para Trabalho Futuro

Com base nos resultados obtidos, propõem-se as seguintes melhorias:

- Explorar técnicas de *oversampling* como SMOTE em alternativa ao *undersampling* utilizado, para preservar mais dados de treino na classificação.
- Explorar modelos de boosting mais avançados como XGBoost ou LightGBM, com melhor escalabilidade.
- Incorporar dados externos de meteorologia e tráfego aéreo para enriquecer os preditores.
- Desenvolver modelos específicos por companhia aérea ou rota.
- Experimentar arquiteturas de *deep learning* temporais (LSTM, Transformer) dado o carácter sequencial dos dados.
- Realizar otimização sistemática de hiperparâmetros via *grid search* ou *Bayesian optimization*.
- Aumentar os recursos computacionais para permitir treino com o dataset completo em todos os modelos.

IV. FASE 6: OPERACIONALIZAÇÃO

A. Modelos Treinados e Artefactos

Todos os modelos finais foram serializados e guardados para permitir reutilização e possível *deployment*:

- Random Forest: `random_forest_model.pkl` (joblib)
- Gradient Boosting: `gradient_boosting_model.pkl` (joblib)
- Decision Tree: `decision_tree_model.pkl` (joblib)
- SVR: `svr_model.pkl` (joblib)
- PyTorch MLP: `deep_learning_model.pt` (torch.save)

Os modelos foram comprimidos numa pasta ZIP para entrega, conforme especificado nos requisitos do projeto.

B. Plano de Deployment

Para implementar os modelos num ambiente de produção, propõe-se o seguinte plano:

- 1) **API de previsão:** expor o modelo Random Forest (melhor desempenho) através de uma API REST desenvolvida em FastAPI ou Flask, recebendo como input as *features* pré-partida e retornando o atraso previsto e a classe de atraso.
- 2) **Pré-processamento em produção:** reproduzir o pipeline de pré-processamento e engenharia de *features* da Parte 1 em tempo real, aplicando os mesmos escalamentos e codificações utilizados no treino.
- 3) **Validação de inputs:** implementar validação dos campos de entrada (datas, códigos de aeroporto, companhia aérea, distância) para garantir que apenas dados válidos chegam ao modelo.
- 4) **Monitorização:** implementar monitorização de *data drift* e degradação de desempenho ao longo do tempo, com re-treino periódico à medida que novos dados operacionais ficam disponíveis.
- 5) **Containerização:** empacotar a aplicação em Docker para garantir portabilidade e reprodutibilidade do ambiente de execução.

V. CONCLUSÃO

Este relatório apresentou a implementação e avaliação completa de múltiplos modelos de *machine learning* para previsão de atrasos em voos domésticos nos EUA, cobrindo as fases 4, 5 e 6 do ciclo de vida de análise de dados.

O projeto demonstrou com sucesso o fluxo completo de *machine learning*: desde o pré-processamento e engenharia de *features* (Parte 1) até à construção, avaliação e operacionalização de modelos (Parte 2). Os resultados confirmam que a previsão de atrasos antes da partida é um problema desafiante, com R^2 baixo em todos os modelos, refletindo a elevada variabilidade operacional e a exclusão estrita de variáveis com *data leakage*. O Random Forest emerge como o modelo mais adequado para *deployment*, pela sua robustez, capacidade preditiva e facilidade de interpretação via importância de *features*.

REFERÊNCIAS

- [1] P. Zel, "Flight Delay and Cancellation Dataset (2019–2023)," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/patrickzel/flight-delay-and-cancellation-dataset-2019-2023/data>